

PYTHON PROGRAMMING

COMPLETE HANDBOOK — BASIC TO ADVANCED

PART 5

Module 14: File Handling

Module 15: Object Oriented Programming

Module 16: Advanced Python

Continues from Parts 1-4 (Modules 1-13)

TABLE OF CONTENTS

TABLE OF CONTENTS	2
MODULE 14: FILE HANDLING	4
14.1 Reading Files	4
14.2 Writing Files	4
14.3 CSV Files	5
14.4 JSON Files	5
Real Project — Simple Note-Taking App	6
Interview Questions — Module 14	6
MCQs — Module 14	6
Exercises — Module 14	7
MODULE 15: OBJECT ORIENTED PROGRAMMING	8
15.1 Class	8
15.2 Object	8
15.3 Constructor	8
15.4 Encapsulation	9
15.5 Inheritance	9
15.6 Polymorphism	10
15.7 Abstraction	10
UML-Style Diagram — Inheritance Hierarchy	10
The Four Pillars of OOP — Summary	11
Interview Questions — Module 15	11
MCQs — Module 15	12
Exercises — Module 15	12
MODULE 16: ADVANCED PYTHON	14
16.1 Iterators	14
16.2 Generators	14
16.3 Decorators	14
16.4 Regular Expressions	15
16.5 Date and Time	16
16.6 Collections Module	16
16.7 OS Module	16
16.8 Sys Module	17
Interview Questions — Module 16	17
MCQs — Module 16	17
Exercises — Module 16	18

QUICK REVISION NOTES — MODULES 14-16	19
COMMON ERRORS AND FIXES — MODULES 14-16	20
CODING BEST PRACTICES — MODULES 14-16	21

MODULE 14: FILE HANDLING

File handling lets a program store data permanently on disk and read it back later — essential for anything that needs to persist beyond a single run, such as logs, reports, or saved settings.

14.1 Reading Files

```
# Recommended: use "with" so the file auto-closes
with open("notes.txt", "r") as f:
    content = f.read()
    print(content)

# Reading line by line
with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip())
```

Mode	Meaning	Example	Result
'r'	Read (default); error if file missing	open(f,"r")	text handle
'w'	Write; creates file or ERASES existing content	open(f,"w")	empty/overwritten file
'a'	Append; adds to the end without erasing	open(f,"a")	existing + new content
'r+'	Read and write	open(f,"r+")	read/write handle

Why use 'with'?

The 'with' statement automatically closes the file when the block ends, even if an error occurs inside it — this prevents resource leaks and is the recommended way to handle files in Python.

14.2 Writing Files

```
with open("notes.txt", "w") as f:
    f.write("Hello, this is line 1.\n")
    f.write("This is line 2.\n")

# Appending instead of overwriting
with open("notes.txt", "a") as f:
    f.write("This line is appended.\n")
```

Common Beginner Mistake

Opening a file with mode 'w' instantly erases all existing content, even before you write anything new. Use mode 'a' (append) if you want to keep what was already there.

14.3 CSV Files

CSV (Comma-Separated Values) files store tabular data as plain text — each line is a row, values separated by commas. Python's built-in csv module handles reading and writing them safely.

```
import csv

# Writing a CSV file
with open("students.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Marks"])
    writer.writerow(["Riya", 88])
    writer.writerow(["Aman", 72])

# Reading a CSV file
with open("students.csv", "r") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Output:

```
['Name', 'Marks']
['Riya', '88']
['Aman', '72']
```

14.4 JSON Files

JSON (JavaScript Object Notation) is a widely used text format for structured data — Python dictionaries convert to and from JSON very naturally, making it the standard format for web APIs and config files.

```
import json

data = {"name": "Riya", "age": 21, "skills": ["Python", "SQL"]}

# Writing JSON to a file
with open("data.json", "w") as f:
    json.dump(data, f)

# Reading JSON from a file
with open("data.json", "r") as f:
    loaded = json.load(f)
    print(loaded["name"])
```

Output:

```
Riya
```

Real Project — Simple Note-Taking App

```
def add_note(note):
    with open("mynotes.txt", "a") as f:
        f.write(note + "\n")

def show_notes():
    with open("mynotes.txt", "r") as f:
        for i, line in enumerate(f, 1):
            print(f"{i}. {line.strip()}")

add_note("Buy groceries")
add_note("Finish Python assignment")
show_notes()
```

Output:

1. Buy groceries
2. Finish Python assignment

Interview Questions — Module 14

Q1. Why should you use the 'with' statement when working with files?

Ans: It automatically closes the file once the block finishes, even if an exception is raised inside it, preventing resource leaks and file corruption — this is the recommended, safest way to work with files.

Q2. What is the difference between 'w' and 'a' file modes?

Ans: 'w' opens a file for writing and immediately erases its existing content (or creates it if it doesn't exist). 'a' opens for appending, adding new content to the end while preserving what was already there.

Q3. How would you read a large file without loading it entirely into memory?

Ans: Iterate over the file object line by line with a for loop (for line in f:), which reads one line at a time instead of using f.read() which loads the whole file at once.

Q4. What module is used to work with JSON data in Python, and what do dump/load do?

Ans: The json module. json.dump()/json.dumps() convert a Python object into JSON (to a file/string). json.load()/json.loads() convert JSON back into a Python object.

Q5. Why is newline="" used when opening a CSV file for writing?

Ans: On Windows, without it, the csv module can insert extra blank lines between rows due to how line endings are translated; newline="" avoids that issue.

MCQs — Module 14

No.	Question	Answer	Options (A-D)
1	Which mode erases existing file content?	B	A) 'r' B) 'w' C) 'a' D) 'r+'
2	Which statement auto-closes a file after use?	C	A) close() B) auto C) with D) finally
3	Which module handles JSON in Python?	A	A) json B) csv C) os D) io
4	json.load() is used to:	D	A) Write to file B) Convert dict to string C) Delete a file D) Read JSON from a file into Python

No.	Question	Answer	Options (A-D)
5	Which mode adds data without erasing existing content?	C	A) 'w' B) 'r' C) 'a' D) 'x'
6	Which module handles CSV files?	B	A) json B) csv C) os D) re

Exercises — Module 14

1. Write a program to count the number of lines, words, and characters in a text file.
2. Write a program to copy the contents of one text file into another.
3. Write a program to store 5 student records (name, marks) into a CSV file, then read and display them.
4. Write a program that saves a Python dictionary of settings into a JSON file and loads it back.
5. Write a program to search for a specific word in a text file and print the line numbers where it appears.

MODULE 15: OBJECT ORIENTED PROGRAMMING

Object-Oriented Programming (OOP) organises code around objects — real-world entities that bundle together data (attributes) and behaviour (methods). Python supports full OOP through classes.

15.1 Class

A class is a blueprint for creating objects. It defines what attributes and methods every object of that type will have — it does not itself store real data until an object is created from it.

```
class Car:
    """Blueprint for a car."""
    def drive(self):
        print("The car is driving")
```

15.2 Object

An object is a specific instance created from a class — it has real, actual data following the class's blueprint.

```
my_car = Car()  # my_car is an object (instance) of Car
my_car.drive()
```

Output:

The car is driving

Real-World Analogy

A class is like an architectural blueprint for a house. An object is an actual house built from that blueprint — you can build many different houses (objects) from the same single blueprint (class).

15.3 Constructor

`__init__()` is a special method automatically called when a new object is created — used to initialise the object's attributes with starting values. `self` refers to the specific object being created/used.

```
class Car:
    def __init__(self, brand, speed):
        self.brand = brand      # instance attribute
        self.speed = speed

    def show_info(self):
        print(f"{self.brand} can go up to {self.speed} km/h")

car1 = Car("Toyota", 180)
car2 = Car("Ferrari", 340)
car1.show_info()
car2.show_info()
```


Output:

```
Toyota can go up to 180 km/h
Ferrari can go up to 340 km/h
```

15.4 Encapsulation

Encapsulation means bundling data and the methods that operate on it inside one class, and restricting direct outside access to some of that data — protecting an object's internal state. Python uses a single leading underscore (`_var`) as a 'protected' convention, and a double leading underscore (`__var`) to trigger name-mangling for stronger privacy.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # private attribute

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

acc = BankAccount(1000)
acc.deposit(500)
print(acc.get_balance())
# print(acc.__balance)    # AttributeError – not directly accessible
```

Output:

```
1500
```

15.5 Inheritance

Inheritance lets a new class (child/derived) reuse the attributes and methods of an existing class (parent/base), while also being able to add or override behaviour of its own.

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name} makes a sound")

class Dog(Animal):    # Dog inherits from Animal
    def speak(self):    # overrides the parent's method
        print(f"{self.name} barks")

a = Animal("Generic Animal")
d = Dog("Rex")
a.speak()
d.speak()
```

Output:

```
Generic Animal makes a sound
Rex barks
```

15.6 Polymorphism

Polymorphism ("many forms") means the same method name behaves differently depending on which object calls it — as shown above, both Animal and Dog have a speak() method, but each produces different behaviour.

```
animals = [Animal("Generic"), Dog("Rex")]
for a in animals:
    a.speak()    # same method call, different behaviour per object
```

Output:

```
Generic makes a sound
Rex barks
```

15.7 Abstraction

Abstraction means hiding complex implementation details and exposing only the essential features to the user. Python achieves formal abstraction using the abc module to define abstract base classes with methods that MUST be implemented by any subclass.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass    # no implementation here – forces subclasses to define it

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius ** 2

c = Circle(5)
print(c.area())
# s = Shape()    # TypeError – cannot instantiate an abstract class
```

Output:

```
78.5
```

UML-Style Diagram — Inheritance Hierarchy

```

+-----+
|   Animal   |
+-----+
| - name     |
+-----+
| + speak() |
+-----+
      ^
      | inherits
+-----+
|   Dog      |
+-----+
| (inherits name) |
+-----+
| + speak() [override] |
+-----+

```

The Four Pillars of OOP — Summary

Pillar	Real-World Meaning
Encapsulation	A capsule/pill hides its contents inside a shell — a class hides its internal data behind methods, protecting it from unwanted direct changes.
Inheritance	A child inherits traits from a parent — a subclass automatically gets the parent class's attributes and methods, and can add its own.
Polymorphism	The same action performed differently by different things — pressing 'play' behaves differently on a music app vs a video app, but the action name is the same.
Abstraction	A car's accelerator hides the engine's complexity — you press one pedal without needing to know how combustion works internally.

Interview Questions — Module 15

Q1. What is the difference between a class and an object?

Ans: A class is a blueprint/template defining attributes and methods. An object is a concrete instance created from that class, holding actual data following the class's structure. You can create many objects from one class.

Q2. What is the purpose of `__init__` and `self`?

Ans: `__init__` is the constructor, automatically called when a new object is created, used to initialise its attributes. `self` refers to the specific instance the method is being called on, letting each object keep its own separate data.

Q3. What is the difference between method overriding and method overloading?

Ans: Overriding is when a child class redefines a method already defined in its parent class, changing its behaviour. Python does not support true method overloading (multiple methods with the same name but different parameters) the way Java/C++ do — default arguments or `*args` are used instead.

Q4. Explain encapsulation with an example.

Ans: Encapsulation bundles data and methods together and restricts direct access to some data. E.g., a `BankAccount` class keeps `__balance` private and only allows changes through `deposit()/withdraw()` methods, preventing invalid direct modification like `acc.__balance = -500`.

Q5. What is the difference between abstraction and encapsulation?

Ans: Abstraction hides implementation complexity and shows only essential features (what something does). Encapsulation hides data and restricts direct access (how it's protected). Abstraction is about design/interface; encapsulation is about data protection.

Q6. Can you create an object of an abstract class?

Ans: No — Python raises a `TypeError` if you try to instantiate a class that has any unimplemented `@abstractmethod`. A subclass must implement all abstract methods before it can be instantiated.

Q7. What is polymorphism, and give a real-life example.

Ans: Polymorphism means the same interface/method behaves differently depending on the object. Real-life example: pressing a car's brake pedal produces the same action name ('brake') across different car models, but each model's internal mechanism differs.

MCQs — Module 15

No.	Question	Answer	Options (A-D)
1	A class is best described as:	A	A) A blueprint for objects B) An actual object C) A function D) A loop
2	Which special method acts as a constructor?	C	A) <code>__new__</code> B) <code>__start__</code> C) <code>__init__</code> D) <code>__construct__</code>
3	<code>self</code> refers to:	D	A) The class itself B) A global variable C) The parent class D) The current object instance
4	A double leading underscore (<code>__var</code>) is used for:	B	A) Constants B) Name-mangled 'private' attributes C) Global variables D) Class methods
5	Inheritance allows a class to:	C	A) Delete another class B) Run faster C) Reuse another class's attributes/methods D) Become abstract automatically
6	Overriding a method means:	A	A) Redefining a parent method in a child class B) Deleting a method C) Calling a method twice D) Making a method private
7	Which module provides tools for abstract classes?	D	A) <code>os</code> B) <code>sys</code> C) <code>collections</code> D) <code>abc</code>
8	Which OOP pillar is demonstrated when one method name behaves differently across classes?	B	A) Encapsulation B) Polymorphism C) Abstraction D) Constructor

Exercises — Module 15

1. Create a class `Student` with attributes `name` and `marks`, and a method to display them.
2. Create a class `Rectangle` with methods to calculate area and perimeter.
3. Create a base class `Employee` and a derived class `Manager` that adds a `bonus` attribute.
4. Create a `BankAccount` class with private `balance` and `deposit/withdraw` methods that validate amounts.

5. Create an abstract class `Shape` with an abstract method `area()`, then implement it in `Circle` and `Square` subclasses.
6. Demonstrate polymorphism using two classes, `Cat` and `Dog`, each with a differently-behaving `sound()` method.

MODULE 16: ADVANCED PYTHON

This module covers Python features that go beyond the basics — tools that make code more efficient, expressive, and professional.

16.1 Iterators

An iterator is an object that produces values one at a time, remembering its position, via the `__iter__()` and `__next__()` methods. Every for loop in Python actually uses an iterator behind the scenes.

```
nums = [1, 2, 3]
it = iter(nums)      # get an iterator from the list
print(next(it))
print(next(it))
print(next(it))
# print(next(it))    # StopIteration – no more items
```

Output:

```
1
2
3
```

16.2 Generators

A generator is a special function that produces a sequence of values lazily (one at a time, on demand) using `yield` instead of `return`. This saves memory since values aren't all stored at once.

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i
        i += 1

for num in count_up_to(5):
    print(num)
```

Output:

```
1
2
3
4
5
```

Generators vs Normal Functions

A normal function computes everything and returns it all at once, using memory for the whole result. A generator pauses at each `yield` and resumes only when the next value is requested — ideal for very large or infinite sequences.

16.3 Decorators

A decorator is a function that wraps another function to add extra behaviour before or after it runs, without changing the original function's code — a very common pattern for logging, timing, or access control.

```
def my_decorator(func):
    def wrapper():
        print("Before the function runs")
        func()
        print("After the function runs")
    return wrapper

@my_decorator
def greet():
    print("Hello!")

greet()
```

Output:

```
Before the function runs
Hello!
After the function runs
```

16.4 Regular Expressions

Regular expressions (regex) describe a search pattern used to find, match, or replace text — handled by Python's built-in `re` module.

```
import re

text = "My phone number is 98765-43210"
match = re.search(r"\d{5}-\d{5}", text)
print(match.group())

emails = re.findall(r"[\w.]+@[ \w.]+", "Contact: a@x.com, b@y.com")
print(emails)
```

Output:

```
98765-43210
['a@x.com', 'b@y.com']
```

Pattern	Meaning	Example	Matches
<code>\d</code>	Any digit (0-9)	<code>\d{3}</code>	matches '123'
<code>\w</code>	Any word character	<code>\w+</code>	matches 'hello'
<code>\s</code>	Any whitespace	<code>\s</code>	matches a space/tab
<code>+</code>	One or more of preceding	<code>a+</code>	'a', 'aa', 'aaa'...
<code>*</code>	Zero or more of preceding	<code>a*</code>	'', 'a', 'aa'...

Pattern	Meaning	Example	Matches
<code>^\$</code>	Start / end of string	<code>^Hi</code>	matches only at start

16.5 Date and Time

```
from datetime import datetime, timedelta

now = datetime.now()
print(now.strftime("%d-%m-%Y %H:%M"))

tomorrow = now + timedelta(days=1)
print(tomorrow.strftime("%d-%m-%Y"))
```

Output:

```
08-09-2026 14:30
09-09-2026
```

16.6 Collections Module

Provides specialised, high-performance container data types beyond the built-in list/dict/set/tuple.

Type	Purpose	Example	Result
Counter	Counts hashable items	<code>Counter("aabbbc")</code>	<code>{'b':3, 'a':2, 'c':1}</code>
defaultdict	Dict with automatic default values	<code>defaultdict(int)</code>	missing key → 0
namedtuple	Tuple with named fields	<code>namedtuple('P', ['x', 'y'])</code>	<code>p.x</code> , <code>p.y</code> access
deque	Fast additions/removals at both ends	<code>deque([1,2,3])</code>	<code>appendleft()</code> , <code>popleft()</code>

```
from collections import Counter

words = "the cat sat on the mat the cat ran".split()
print(Counter(words))
```

Output:

```
Counter({'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1, 'ran': 1})
```

16.7 OS Module

Lets Python interact with the operating system — file paths, directories, environment variables.


```
import os

print(os.getcwd())      # current working directory
os.mkdir("new_folder")  # create a folder
print(os.listdir("."))  # list files in current directory
print(os.path.exists("notes.txt"))
```

16.8 Sys Module

Provides access to variables and functions closely tied to the Python interpreter itself, such as command-line arguments and exiting a script.

```
import sys

print(sys.version)      # Python version being used
print(sys.argv)         # command-line arguments passed to the script
# sys.exit()            # exits the program immediately
```

Interview Questions — Module 16

Q1. What is the difference between an iterator and an iterable?

Ans: An iterable is any object you can loop over (like a list or string) that implements `__iter__()`. An iterator is the object actually returned by `iter()`, which implements `__next__()` to produce values one at a time and remembers its position.

Q2. What is the difference between yield and return?

Ans: `return` sends back a value and ends the function completely. `yield` sends back a value but pauses the function's state, allowing it to resume from that exact point the next time a value is requested — this is what makes generators memory-efficient.

Q3. What is a decorator, and give a real-world use case?

Ans: A decorator is a function that wraps another function to add extra behaviour without modifying its source code, applied using the `@` syntax. Common real-world uses include logging function calls, timing execution, and enforcing login/authentication checks.

Q4. Why are generators more memory-efficient than lists for large sequences?

Ans: A generator produces values one at a time on demand and never stores the whole sequence in memory, unlike a list which holds every element at once — critical when processing very large or infinite data streams.

Q5. What module would you use to search for patterns in text, and name one useful pattern.

Ans: The `re` module. `\d` matches a digit, `\w` matches a word character — useful, for example, in `re.findall(r'\d+', text)` to extract all numbers from a string.

Q6. What does `collections.Counter` do?

Ans: It counts occurrences of each unique, hashable element in an iterable and returns a dictionary-like object mapping each item to its count, sorted by frequency when printed.

MCQs — Module 16

No.	Question	Answer	Options (A-D)
-----	----------	--------	---------------

No.	Question	Answer	Options (A-D)
1	Which keyword is used to create a generator function?	C	A) return B) generate C) yield D) iter
2	Which symbol is used to apply a decorator?	B	A) # B) @ C) & D) \$
3	Which module provides regular expressions?	A	A) re B) regex C) pattern D) match
4	Counter({'a':2,'b':1}) counts occurrences in a/an:	D	A) String only B) File C) Dictionary only D) Any iterable
5	next() on an exhausted iterator raises:	B	A) IndexError B) StopIteration C) ValueError D) TypeError
6	Which module gives access to command-line arguments?	C	A) os B) re C) sys D) json
7	deque is most useful for:	A	A) Fast operations at both ends B) Sorting only C) JSON parsing D) File reading

Exercises — Module 16

1. Write a generator function that yields the Fibonacci sequence up to n terms.
2. Write a decorator that prints how long a function takes to run.
3. Use regex to validate whether a given string is a valid email address.
4. Use Counter to find the most common word in a paragraph.
5. Write a program using the os module to list all .txt files in the current directory.
6. Use datetime to calculate how many days remain until a given future date.

QUICK REVISION NOTES — MODULES 14-16

- Always use 'with open(...) as f:' for file handling — it auto-closes the file safely.
- File modes: 'r' read, 'w' write (erases!), 'a' append. csv and json modules handle structured file formats.
- Class = blueprint, Object = actual instance. `__init__()` initialises attributes; `self` refers to the current object.
- Four OOP pillars: Encapsulation (hide/protect data), Inheritance (reuse via parent-child), Polymorphism (same method, different behaviour), Abstraction (hide complexity, expose essentials).
- Double underscore (`__var`) triggers name-mangling for a 'private' attribute; `abc.ABC` + `@abstractmethod` enforce abstraction.
- Generators (`yield`) produce values lazily and save memory versus building a full list upfront.
- Decorators (`@decorator`) wrap a function to add behaviour without changing its source code.
- `re` module handles pattern matching (`\d` digits, `\w` word chars, `\s` whitespace); `collections` module offers `Counter`, `defaultdict`, `namedtuple`, `deque`.

COMMON ERRORS AND FIXES — MODULES 14-16

Error	Cause	Fix
FileNotFoundError	Opening a file in 'r' mode that doesn't exist	Check the path, or use 'w'/'a' to create it
Data lost after opening a file	Used 'w' mode instead of 'a'	Use 'a' to append instead of overwrite
TypeError: Can't instantiate abstract class	Not all @abstractmethod methods implemented	Implement every abstract method in the subclass
AttributeError on private attribute	Accessing __var directly from outside the class	Use a public getter method instead
StopIteration	Calling next() after an iterator is exhausted	Use a for loop, which handles this automatically

CODING BEST PRACTICES — MODULES 14-16

- Always use 'with' for file handling instead of manually calling open()/close().
- Follow class naming convention: CapitalizedWords for class names (PascalCase), lowercase_with_underscores for methods/attributes.
- Favour composition or small focused classes over deep, complicated inheritance hierarchies.
- Use generators instead of lists when working with very large datasets to save memory.
- Keep decorators simple and well-documented — they can make debugging harder if overused or unclear.
- Validate and test regex patterns carefully on real sample data — they can silently match more or less than intended.